SECTION 1

A Quick Tour of TensorFlow

- **Role:** TensorFlow is a numerical and autodiff library optimized for large ML workloads, with optional GPU/TPU execution, graph optimization, and export for deployment beyond Python.
- **Ecosystem:** TensorBoard, TFX, TF Hub, and Model Garden sit beside the core runtime for experiment tracking, production pipelines, and pretrained weights.
- **Stack:** Most work stays in `tf.keras` and `tf.data`, but lower-level APIs expose tensors, variables, and ops when custom logic is required; TensorBoard, TF Serving, and Hub sit on top of the runtime.
- **Execution:** Ops run through a C++ kernel layer with device-specific implementations; the Python API builds computation that the engine schedules and fuses.

SECTION 2

Using TensorFlow like NumPy

- **Tensors:** `tf.constant`, slicing, and elementwise or matrix ops mirror NumPy but run on accelerators; dtypes default to float32 for neural nets.
- **Variables:** `tf.Variable` holds mutable trainable state; update with `assign`, `assign_add`, or indexed `scatter_*`—what optimizers modify during training.
- **Interoperability:** Tensors and NumPy convert bidirectionally; TensorFlow does not silently cast incompatible types—use `tf.cast` deliberately to avoid hidden performance traps.
- **Extra structures:** Sparse, ragged, string, and queue types support non-dense pipelines without abandoning the TensorFlow type system.
- **Queues and generators:** Legacy queue pipelines still appear in older code, but `tf.data` (Chapter 13) is the preferred way to feed custom training loops today.

SECTION 3

Customizing Models and Training Algorithms

- **Losses and metrics:** Implement callables or subclass `keras.losses.Loss` / `keras.metrics.Metric`; streaming metrics (`update_state`, `result`) are required when a batch-wise average misstates quantities like precision.
- **Activations, init, regularizers, constraints:** Simple functions or small subclasses with `get_config` enable save/load via `custom_objects`.
- **Custom layers and models:** Subclass `Layer` with `build` → `call` → `get_config`; nested layers are tracked automatically; subclass `Model` for arbitrary forward logic, skip connections, and `add_loss` / `add_metric` on internal tensors.
- **Saving custom code:** Function-based losses save by name; subclassed components need `get_config` / `from_config` parity for reliable `load_model`.
- **Autodiff:** `tf.GradientTape` records forward ops and computes gradients; `persistent=True` allows multiple backward passes; `tape.watch` and `tf.stop_gradient` control what receives gradients; `@tf.custom_gradient` fixes unstable derivative implementations.

SECTION 4

Custom Training Loops

- **When needed:** `fit()` assumes one optimizer and standard update rules; multi-optimizer setups (e.g., Wide & Deep), exotic gradient surgery, or mandatory transparency justify a manual loop.
- **Pattern:** For each batch, forward under tape, sum main loss plus layer regularization losses, `tape.gradient` on `trainable_variables`, `optimizer.apply_gradients`, update metrics, and honor `training=True` for `BatchNormalization` and Dropout.
- **Trade-off:** Loops offer full control at the cost of verbosity and easy mistakes with weight constraints or regularization terms.
- **Status UX:** Manual loops should aggregate metrics explicitly and reset state each epoch, mimicking what `fit()` handles via callbacks.

SECTION 5

TensorFlow Functions and Graphs

- **`@tf.function`:** Wraps Python functions in traced, optimizable graphs for faster repeated execution and export; Keras often converts custom layers automatically.
- **AutoGraph and tracing:** Control flow is lifted into TensorFlow ops; unique input shapes cache separate graphs—avoid Python scalars that trigger endless re-tracing.
- **Rules:** Prefer TensorFlow ops over NumPy inside functions, create stateful variables outside or on first call, and use `tf.range` when loops must live in the graph; set `dynamic=True` or `run_eagerly=True` when debugging demands eager semantics.
- **Serialization:** Models with custom classes need `custom_objects` maps when loading HDF5/SavedModel artifacts so rebuilt layers match training definitions.
- **Debugging graphs:** Inspecting `tf.autograph` output or running with `run_eagerly=True` clarifies when Python side effects fail to run each step.